

Seat Locking and Concurrency Handling

1. Overview & Locking Approach

In high-concurrency ticket booking platforms like BookMyShow, popular movie releases trigger intense traffic spikes where thousands of users compete for identical seats simultaneously. Using traditional relational database row locks (`SELECT ... FOR UPDATE`) leads to severe connection pool exhaustion and high query latency.

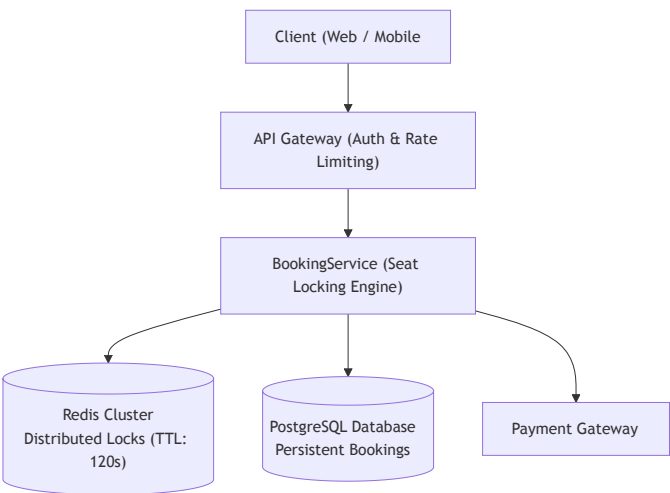
To guarantee zero double-bookings with sub-millisecond latency, we use a **distributed locking mechanism using Redis SETNX** (Set if Not Exists) with an atomic time-to-live (TTL).

Lock Acquisition Logic

```
Algorithm: AcquireSeatLock(show_id, seat_id, user_id)
1. lock_key = "seat:" + show_id + ":" + seat_id
2. acquired = redis.set(lock_key, user_id, NX=True, EX=120)
3. if not acquired:
    raise SeatUnavailableException("Seat already locked")
4. return lock_success
```

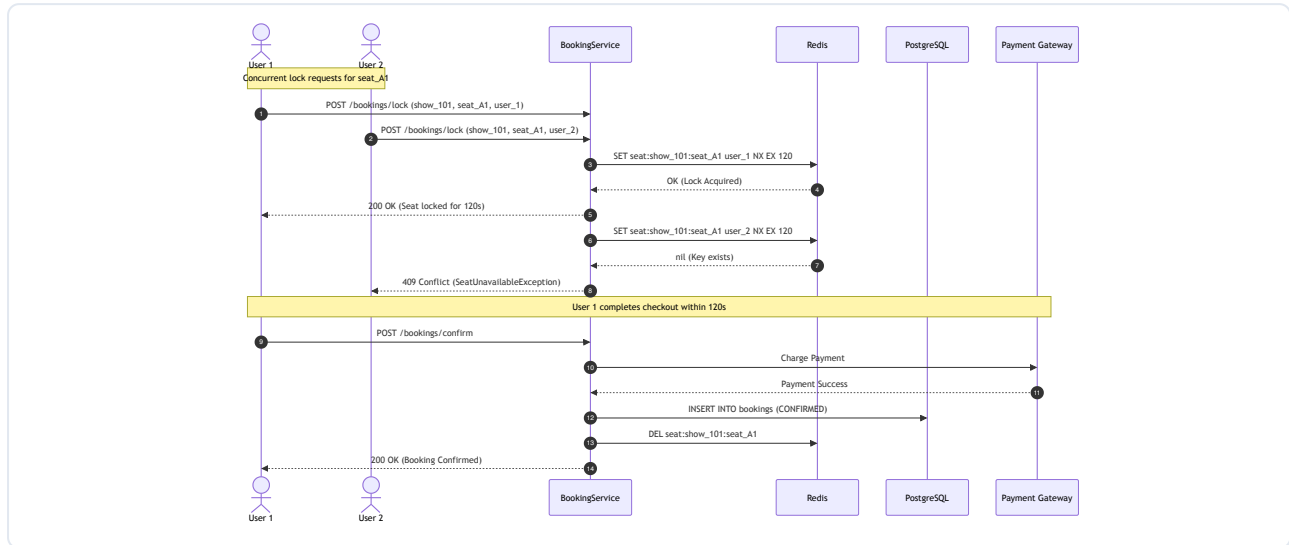
2. System Component Diagram

The Seat Locking engine resides inside `BookingService`, coordinating between client requests, temporary locks in Redis, persistent database storage, and the payment gateway.



3. Concurrency Sequence: Simultaneous Booking Attempt

The diagram below illustrates two users attempting to lock the exact same seat (seat_A1 for show_101) at the exact same millisecond:



4. Idempotency Guarantees

- Idempotency Key:** Each lock request includes a unique client-generated `idempotency_key` (or `booking_id`).
- Deduplication Check:** If network interruption causes the client to retry `POST /bookings/lock`, BookingService inspects the existing Redis key. If the key already exists and holds the same `user_id` and `booking_id`, it returns the existing hold confirmation and remaining TTL rather than returning an error.

5. Lock Timeout & Auto-Release Strategy

- Atomic Expiration (TTL: 120s):** The Redis lock is assigned an explicit 120-second (2 minutes) expiration via `EX 120`. Redis handles eviction automatically in memory without needing background worker threads or cron jobs.
- Checkout Abandonment:** If a user closes the app, navigates away, or lets the countdown timer expire, Redis purges the key after 120 seconds. The seat immediately becomes available to other users.
- Successful Booking Confirmation:** Once payment verification succeeds, the booking record is committed to PostgreSQL as `CONFIRMED`, and the temporary lock is explicitly deleted via `redis.delete(lock_key)`.

6. Handling Partial Failures

- Payment Declined / User Cancellation:** If the payment gateway declines the card or the user cancels checkout, BookingService executes `redis.delete(lock_key)` immediately so others can book the seat without waiting for the 120-second timeout.
- Service or Network Crash:** If BookingService crashes while the user is on the payment screen, the 120-second Redis TTL acts as an autonomous safety boundary, freeing the seat when the timer elapses.
- Payment Succeeded but Database Write Fails:** If payment succeeds but the database write fails, an asynchronous reconciliation worker detects the discrepancy, issues an automated refund via the payment gateway, and releases the Redis lock.